

README-K1

Evan He, e7he, 20946651

Ian Zhao, i6zhao, 20988818

Abstract	3
Clarification	3
Commit and SHA	3
Building	3
Kernel Architecture	4
Components	4
Control Flow	5
Task Creation	5
Context Switching	5
Kernel State	6
Output	7

Abstract

K1 contains a working kernel that's capable of performing context switching, and schedule tasks in a round robin format.

Clarification

1. The kernel can have a maximum of 128 tasks at a time, each with a 2MB of stack frame.
2. There are three task priorities: LOW, MEDIUM, HIGH.
3. The first task id starts at 1, and increases by 1 each allocating a new task. If the id reaches the maximum int value, then the kernel reports it as an error to avoid duplicated task id.
4. If a task's parent has exited, then MyParentTid still returns the original tid of the parent.
5. The task directly created by the kernel has no parent.

Commit and SHA

Repo: https://git.uwaterloo.ca/e7he/cs452_kernel

Commit: 8371d1a80d6bb24cf1e96b838e5d8487a4d69810

Building

The kernel image at "kitty_kernel.img" is built by running "make". There are no special requirements to build the project in the linux.student.cs environment.

Kernel Architecture

Components

TaskDescriptor

TaskDescriptor is the handle to an allocated task. It contains all the meta data such as the taskId, priority, parent, runState, stackPointer, etc.

TaskScheduler

TaskScheduler is a singleton class that provides operations to manipulate the task queues.

StackContext

A placeholder class to define the saved context layout of each user task.

RoundRobinQueue

A circular linked list that iterates through all the items in a round robin format. Internally, it stores a set of TaskDescriptor, and uses TaskDescriptor's intrusive data structure's link field to form an order.

MultiLevelQueue<T>

A template class that holds an array of task queues indexed by task's priority. It is a wrapper class that has the same operations as the internal queue, and selects the corresponding queue based on task's priority.

SlabAllocator<T, SIZE>

A template class that allocates an array of fixed size memory blocks. Internally it uses a free list to track the available memory blocks. Each freed memory block is an intrusive data structure, and are linked together via an internal linkage. When the memory block is allocated, it is reinterpreted as an array of unused bytes.

Control Flow

Task Creation

The kernel create a task with the following procedure:

1. Allocate a stack frame using the SlabAllocator dedicated for stack frames.
2. Allocate a TaskDescriptor using the SlabAllocator dedicated for the task descriptor.
3. Pretend the beginning of the stack frame already contains the task context, including registers x0 - x30, ESR_EL1, SPSR_EL1, ELR_EL1.
4. Initialize the link register to the Exit() system call, ELR_EL1 to the function entry, and all other registers to 0.
5. Initialize the TaskDescriptor's fields, and enqueue into the scheduler queue.

Context Switching

1. In the boot loader code, the boot loader sets the exception vector to a dedicated vector with synchronized exception pointing to a function named svcEntry.
2. When a system call is called, the system call calls SVC #N, where N depends on the type of the system call.
3. SVC #N looks up the exception vector, and jumps to svcEntry and switches to the EL1 mode.
4. svcEntry temporarily switches to EL0 stack pointer, and pushes the current task's context into its dedicated stack. It then switches back to EL1 stack pointer, and calls the syscallEntry(taskStackPointer).
5. syscallEntry(taskStackPointer) is a C function that checks ESR_EL1 to see the svc code, and performs the corresponding system calls operations such as Create, Yid, Yield etc. At the end of the syscallEntry, it schedules the next task in a round robin format.

6. Context switches to the next scheduled task by temporarily switching to EL0 stack pointer, and pops the context registers from the task's stack. Set ELR_EL1 to the PC to jump to the address with ERET.

::Create(priority, function)



SVC #0



svcEntry:

save task context
branch to syscallEntry



syscallEntry:

depends on the svc number, it forwards the call to the function that performs the actual operation, e.g. syscall_handler::Create(priority, function)



call TaskScheduler to schedule the next task.



context switch into the next task

Kernel State

We adopted the “temporary stack” kernel design, where the kernel acts as an event handler. As such, the function call that context switches to a task never returns. And, in svcEntry, the assembly resets the kernel stack pointer back to its initial value. All kernel state that persists between kernel entries is kept as static variables.

Output

1. Kitty kernel version: Jan 25 2026 / 04:05:51
2. Created: 2
3. Created: 3
4. MyTid: 4, MyParentTid: 1
5. MyTid: 4, MyParentTid: 1
6. Created: 4
7. MyTid: 5, MyParentTid: 1
8. MyTid: 5, MyParentTid: 1
9. Created: 5
10. FirstUserTask: exiting
11. MyTid: 2, MyParentTid: 1
12. MyTid: 3, MyParentTid: 1
13. MyTid: 2, MyParentTid: 1
14. MyTid: 3, MyParentTid: 1
15. All tasks exited. Press any key to reboot...

Line 1: Enter kernel main, prints the hello message.

Line 2: The kernel creates task 1, with MEDIUM priority, which in turns creates task 2 with LOW priority. The scheduler selects the next scheduled task and context switch into task 1, and prints the tid.

Line 3: Task 1 creates task 3 with LOW priority. The scheduler selects the next scheduled task and context switch into task 1, and prints the tid.

Line 4: Task 1 creates task 4 with HIGH priority. The scheduler selects the next scheduled and task context switch into task 4, and prints the message.

Line 5: Task 4 yields. The scheduler selects the next scheduled task and context switch into task 4, and prints the message.

Line 6: Task 4 exits. The scheduler selects the next scheduled task and context switch into task 1, and prints the message.

Line 7: Task 1 creates task 5 with HIGH priority. The scheduler selects the next scheduled task and context switch into task 5, and prints the message.

Line 8: Task 5 yields. The scheduler selects the next scheduled task and context switch into task 5, and prints the message.

Line 9: Task 5 exits. The scheduler selects the next scheduled task and context switch into task 1, and prints the message.

Line 10: Task 1 prints the messages.

Line 11: Task 1 exits, and the scheduler selects the next scheduled task and context switch into task 2 over task 3 because it was the very last executed task in the round robin queue.

Line 12 - 14: Each task prints its message, and yields and prints the message then exits. The scheduler rotates between task 2 and task 3 because they have the same and the highest priorities in the round robin queue.

Line 15: The last task, task 3 exits. There's no more tasks in the scheduler queue. Print a custom message and ask the user to reboot.